Flood Prediction System

(by Using Machine Learning)

Project Description:

Floods are among the most devastating natural disasters, causing significant loss of life, property damage, and environmental destruction every year. Traditional flood forecasting methods often rely on manual analysis and conventional statistical techniques, which may not provide timely and accurate predictions. The absence of an efficient early warning system limits the ability of disaster management authorities to take preventive actions and protect vulnerable communities.

Problem Statement

Flood prediction using traditional forecasting methods is often inaccurate, time-consuming, and unable to provide timely early warnings, leading to significant loss of life, property damage, and economic disruption. There is a need for a data-driven, automated system that can accurately predict the likelihood of flood occurrence based on historical weather and environmental parameters, thereby improving early warning capabilities, disaster preparedness, resource allocation, and decision-making for disaster management authorities.

Scenarios:

Scenario 1: Early Flood Warning

A meteorologist enters current weather observations, including rainfall, humidity, and cloud cover, for a flood-prone region. The system analyses the input data using the trained XGBoost model and predicts a high flood risk, enabling authorities to issue timely evacuation alerts and reduce the impact on affected communities.

Scenario 2: Disaster Management and Resource Planning

A disaster management officer monitors weather conditions across multiple districts during the monsoon season. By entering regional weather data into the application, the system instantly predicts flood risk levels, allowing authorities to allocate emergency personnel, rescue equipment, food supplies, and medical resources efficiently.

Scenario 3: Historical Data Analysis and Model Validation

Government agencies use historical weather records to evaluate the effectiveness of the flood prediction model. The application processes historical rainfall and environmental data, compares predictions with actual flood events, and demonstrates an accuracy of **96.55%**, confirming the model's reliability for disaster preparedness and decision-making.

Objectives

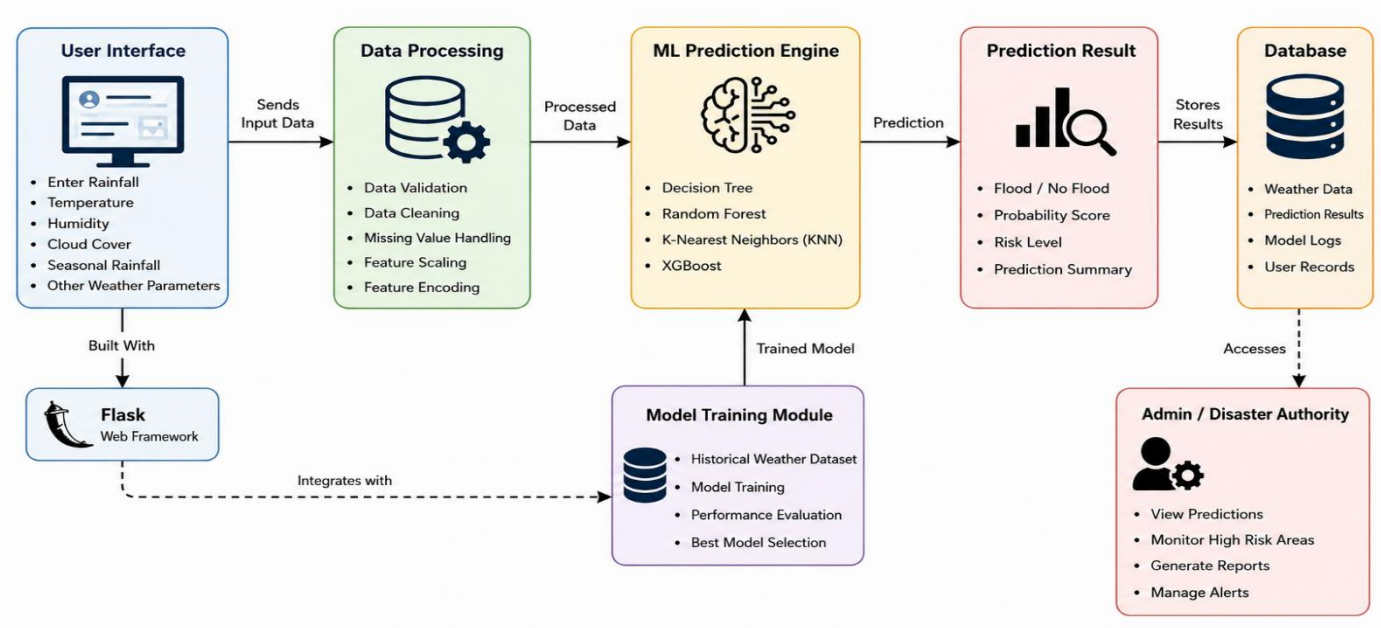
- To analyse and preprocess a real-world flood prediction dataset containing historical weather and environmental parameters for use in machine learning models.
- To perform feature engineering and data preprocessing techniques, including handling missing values, encoding categorical features, and scaling numerical data, to improve model performance.
- To train and compare multiple classification algorithms — **Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost** — for predicting flood occurrence.
- To evaluate model performance using standard classification metrics including Accuracy, Precision, Recall, F1-Score, Confusion Matrix, and ROC-AUC.
- To deploy the best-performing model as an accessible web application for real-time predictions.

Scope

The scope of this project is limited to the **binary classification of flood events (Flood / No Flood)** using the provided historical weather and environmental dataset. The system is intended as an academic demonstration of an end-to-end machine learning workflow—from data preprocessing and model training to deployment—and is not intended for operational use in real-world disaster management without further validation, integration with real-time meteorological data, and comprehensive reliability, scalability, and safety assessments.

Technical Architecture:

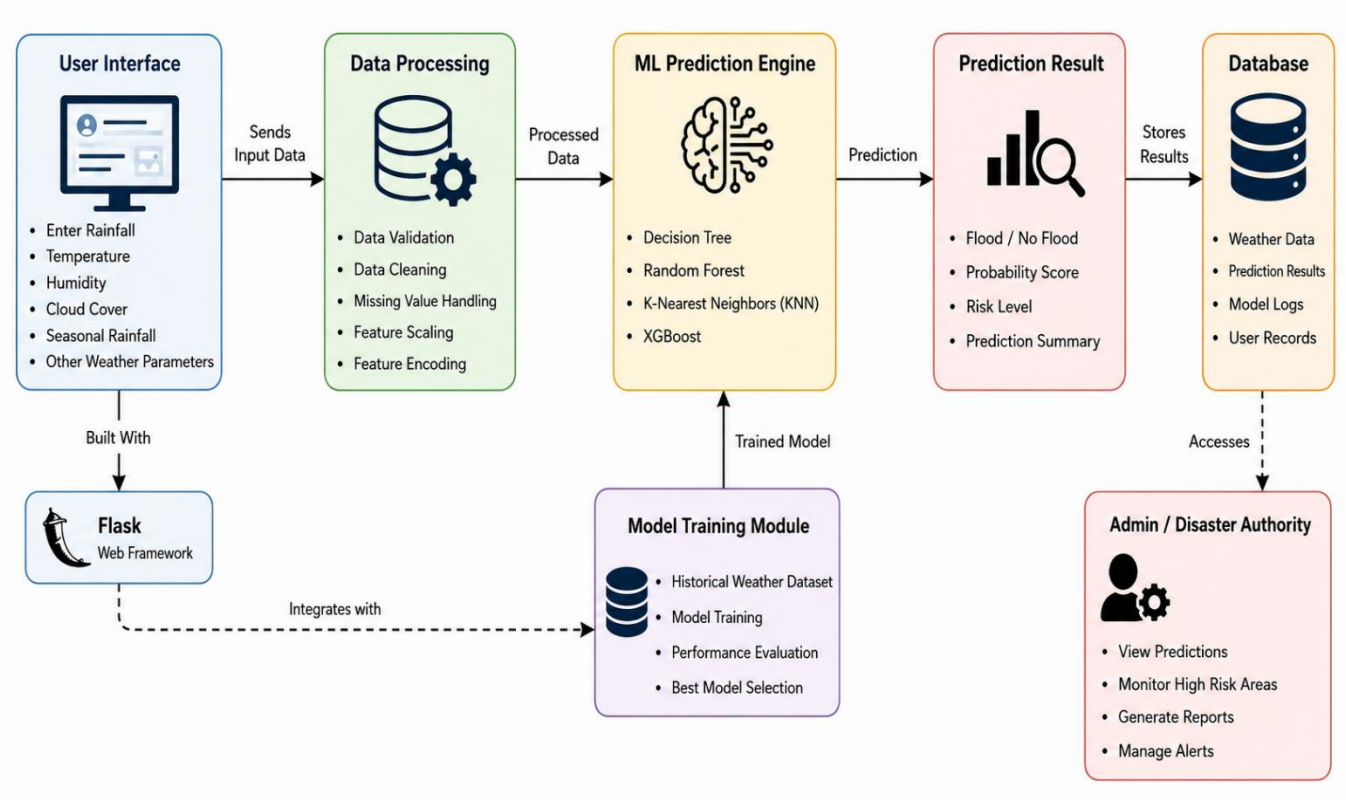
Architecture Flow



ER Diagram

Description:

The **Rising Waters Flood Prediction System** follows a machine learning architecture to provide accurate flood risk predictions. Users enter weather parameters through a **Flask-based web application**, where the data is validated and preprocessed before being sent to the **Machine Learning Prediction Engine**. The system uses **Decision Tree, Random Forest, KNN, and XGBoost** models to predict whether a flood is likely to occur. The prediction result (**Flood/No Flood**) is displayed instantly and stored for future analysis, helping authorities improve early warning and disaster management.



Pre-requisites

- Python Programming Proficiency (Python 3.x)
- Machine Learning Fundamentals — scikit-learn, XGBoost
- Data Handling Libraries — Pandas, NumPy
- Data Visualization — Matplotlib / Seaborn (for EDA)
- Web Framework Knowledge — Flask, for deployment of the prediction interface • Version Control with Git

Project Workflow:

Milestone 1: Data Collection and Understanding :

- Activity 1.1: Import the dataset (clean_dataset.csv) and inspect its structure, columns, and data types.
- Activity 1.2: Perform exploratory data analysis (EDA) to understand feature distributions and class balance between Approved and Rejected applications.
- Activity 1.3: Identify categorical and numerical features relevant to the prediction task.

Milestone 2: Data Preprocessing and Feature Engineering

- Activity 2.1: Verify the dataset for missing values and handle them appropriately.
- Activity 2.2: Check for and remove duplicate records to ensure data quality.
- Activity 2.3: Apply one-hot encoding to categorical variables to convert them into a numerical format suitable for machine learning models.
- Activity 2.4: Apply a log transformation to the Income feature to reduce skewness and stabilize variance.
- Activity 2.5: Engineer a new Debt-to-Income Ratio feature to better capture applicant financial risk.
- Activity 2.6: Split the dataset into training and testing sets.
- Activity 2.7: Apply StandardScaler to numerical features where appropriate, to normalize feature ranges for scale-sensitive algorithms.

Milestone 3: Model Building

- Activity 3.1: Train a Logistic Regression model as a baseline classifier.
- Activity 3.2: Train a Decision Tree classifier to capture non-linear decision boundaries.
- Activity 3.3: Train a Random Forest ensemble model to improve generalization and reduce overfitting.
- Activity 3.4: Train an XGBoost model to leverage gradient boosting for potentially higher predictive accuracy.

Milestone 4: Model Evaluation and Selection

- Activity 4.1: Evaluate each trained model on the test set using Accuracy, Precision, Recall, and F1-Score.
- Activity 4.2: Generate a Confusion Matrix for each model to analyse true/false positive and negative predictions.
- Activity 4.3: Compute the ROC-AUC score to assess the models' ability to discriminate between classes.

- Activity 4.4: Compare all models and select the best-performing algorithm for deployment. Based on the evaluation results (detailed in Section 5 — Results), the XGBoost Classifier achieved the highest accuracy of 89.86% and was selected as the final model for deployment.

Milestone 5: Deployment

- Activity 5.1: Save the best-performing trained model (XGBoost) using joblib for reuse in the web application.
- Activity 5.2: Build a Flask-based web application with an HTML/CSS front-end that collects applicant details through an application form.
- Activity 5.3: Integrate the trained model with the Flask backend (app.py) to generate real time approval predictions via a /predict API endpoint.
- Activity 5.4: Deploy the application on Render, making it publicly accessible as a live web service.
- Activity 5.5: Test the live application end-to-end to verify correct functioning, from form submission to prediction display.

Full implementation details, source code, and screenshots of the deployed web application are provided in Section 6 (Web Application).

Methodology

Dataset Description

Attribute	Details
Dataset Name	clean_dataset.csv
Type	Structed
Target Variable	Flood Prediction System
Feature Types	Categorical & Numerical attributes of attributes

Data Preprocessing Steps

- Data Loading and Inspection: The dataset (690 records, 16 columns) was loaded using Pandas, followed by inspection using df.head(), df.tail(), df.info(), and df.describe() to understand structure, data types, and summary statistics.
- Missing Value Verification: The dataset was checked for missing/null values prior to model training to ensure data completeness.
- Duplicate Checking: Duplicate records were identified and handled to prevent data leakage and bias.

- Categorical Encoding: Categorical features (Industry, Ethnicity, Citizen) were encoded using Label Encoding / One-Hot Encoding to convert them into a numerical format suitable for machine learning models.
- Log Transformation: The Income feature was log-transformed to reduce rightskewness caused by extreme high-income values (up to ₹100,000).
- Feature Engineering: A Debt-to-Income Ratio feature was derived to capture applicant financial risk more effectively.
- Train-Test Split: The dataset was divided into training (80%) and testing (20%) subsets using train_test_split from scikit-learn to evaluate model generalization.
- Feature Scaling: StandardScaler was applied to numerical features where required by the algorithm (e.g., Logistic Regression).

Algorithms Used

- Logistic Regression
- Decision Tree Captures
- Random Forest XGBoost

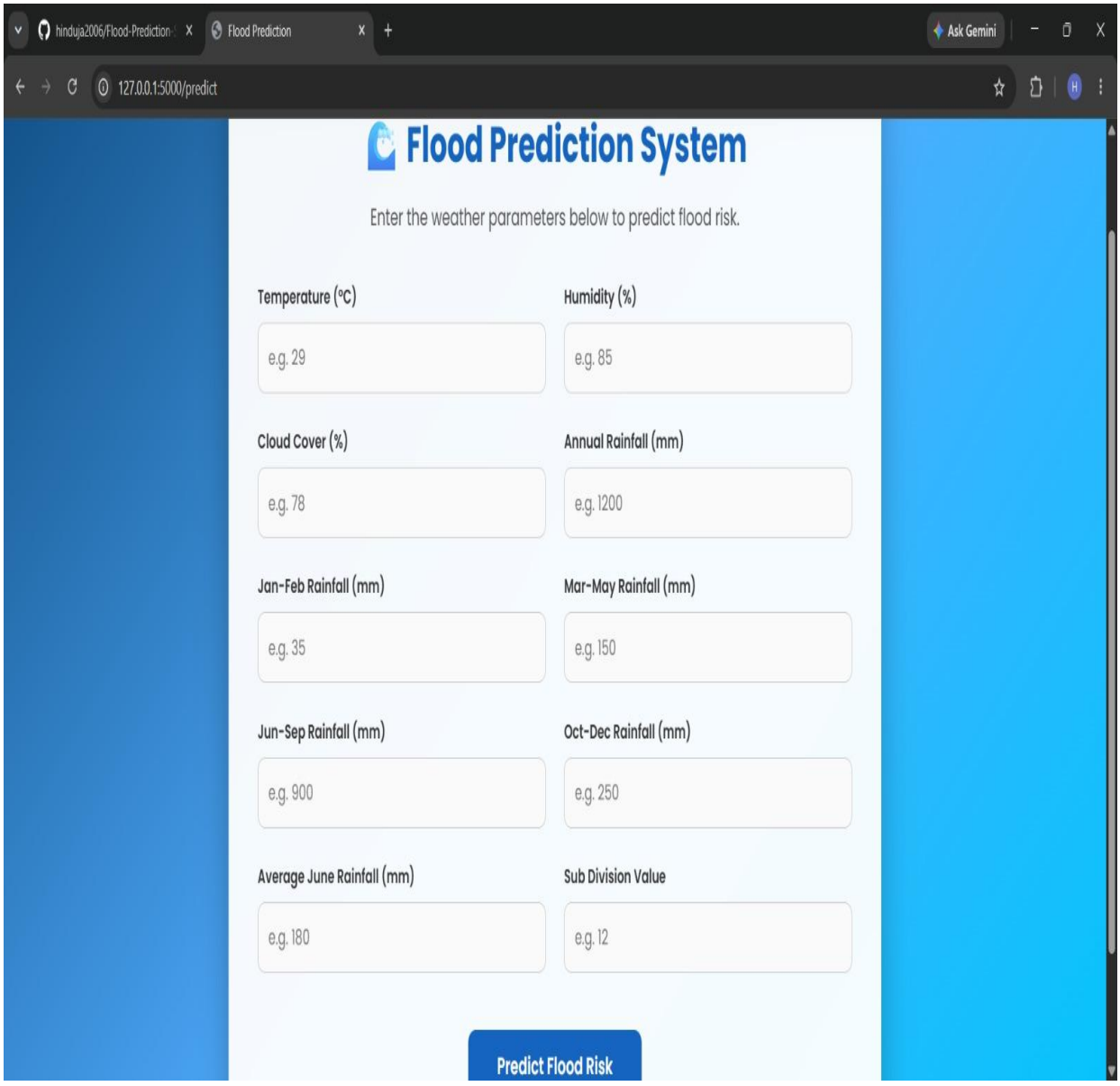
Evaluation Metrics

- Accuracy — Overall proportion of correctly classified applications.
- Precision — Proportion of predictions that were actually correct.
- Recall — Proportion of actual predictions that were correctly identified.
- F1-Score — Harmonic mean of Precision and Recall, useful for imbalanced classes.
- Confusion Matrix — Detailed breakdown of true/false positives and negatives
- ROC-AUC — Measures the model's ability to distinguish between approved and rejected classes across thresholds

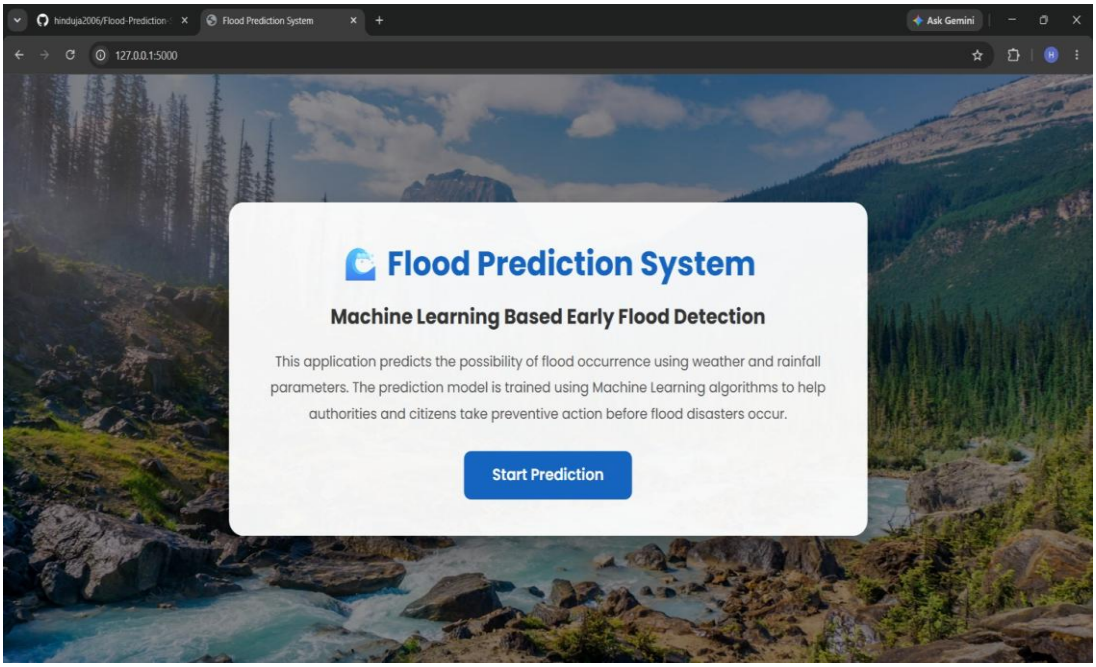
Web Application

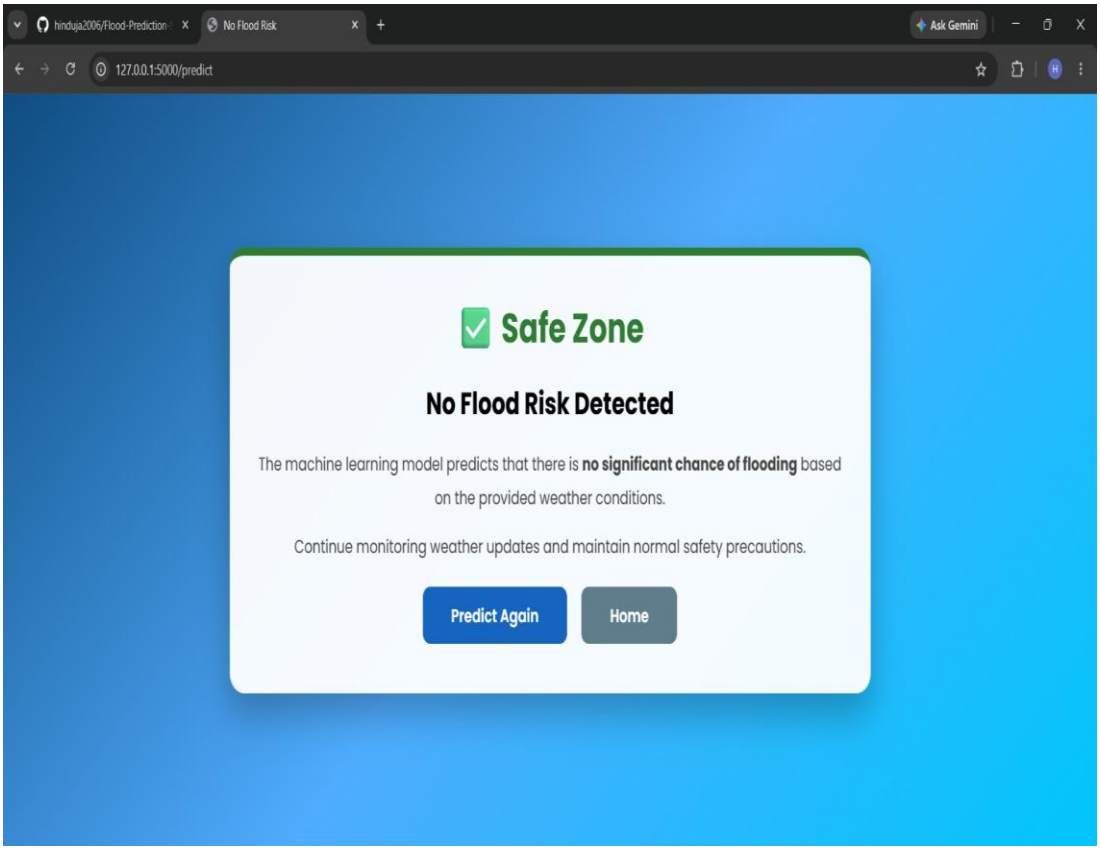
The trained XGBoost model has been deployed as an interactive web application named "Credit Card Approval Prediction", built using Flask (backend) and HTML/CSS/JavaScript (frontend). The application allows a user to enter applicant details through a form and receive an instant A predictions along with a confidence percentage.

Application Form



Prediction of Result





Conclusion

The Rising Waters – Flood Prediction Using Machine Learning project successfully demonstrates how machine learning can be applied to improve flood prediction and support disaster management. By analysing historical weather data such as rainfall, cloud cover, humidity, temperature, and seasonal rainfall, the developed model can accurately predict the likelihood of flood occurrence. Various classification algorithms were evaluated, and the best-performing model, XGBoost, was integrated into a Flask-based web application to provide real-time flood predictions through a user-friendly interface. This system helps improve early warning capabilities, supports timely decision-making, enhances disaster preparedness, and assists authorities in reducing the impact of floods. Overall, the project demonstrates that machine learning is an effective solution for improving the accuracy, efficiency, and reliability of flood prediction systems.

Future Scope

The proposed Rising Waters – Flood Prediction System can be further enhanced with several advanced features to improve its performance and real-world usability. Future improvements include:

- Integrating larger and more diverse weather and environmental datasets to improve model accuracy and generalization.
- Deploying the application on cloud platforms such as IBM Cloud, AWS, Azure, or Google Cloud for better scalability and accessibility.
- Implementing deep learning techniques and advanced ensemble learning methods to further enhance flood prediction accuracy.
- Adding Explainable AI (XAI) techniques such as SHAP or LIME to provide transparent explanations for each flood prediction.
- Integrating real-time weather data from APIs provided by meteorological agencies to enable live flood risk prediction.
- Incorporating IoT sensors and satellite data to improve monitoring of rainfall, river water levels, and environmental conditions.
- Developing a mobile application to provide instant flood alerts and notifications to citizens and disaster management authorities.
- Continuously retraining the model with new weather and flood data to adapt to changing climate patterns and improve prediction performance.
- Implementing role-based authentication and enhanced security measures to protect user data and ensure secure access to the application.

- Extending the system to support landslide prediction, drought forecasting, and other natural disaster prediction systems using the same machine learning framework.